

Swissbit PQC-evaluator

Version 1.0

We have implemented

- support for ML-DSA based credentials
 - authenticatorMakeCredential with pubKeyCredParams including any ML-DSA
- only direct self-attestation
 - uses credential private key for attestation

Restrictions

- PQ-credentials can only be created or used when connected over USB

Possibly later via updates

- support for ML-DSA keys in PIV applet
 - proprietary; no PIV algorithm identifiers yet
- device attestation based on ML-DSA
- hybrid EC-DSA + ML-DSA credentials and attestation
 - proprietary; no IANA COSE identifiers yet
 - EC-DSA keys on certified secure element
- new crypto agile PIN/UV Auth Protocol with PQ-secure option
 - proprietary; no CTAP specification yet
 - would require changes to host CTAP implementation

RP requirements

The RP SHALL support ML-DSA verification (default “hedged” variant) with all three parameter sets. For example, we verified compatibility with node.js v24.14.0 WebCrypto "ML-DSA-44", "ML-DSA-65", and "ML-DSA-87" (experimental).

`navigator.credentials.create()`

Caution

The ML-DSA private credential key will be stored and processed in an uncertified secure element.

This private key will never leave this secure element, and the secure element is designed to withstand hardware level attacks on the private key (designed for CC EAL4+ according to BSI-CC-PP-0084-2014).

However, we cannot give any guarantees as the secure element is not yet certified.

Hence, these credentials should be used with care, considering the risk of storage on uncertified hardware.

Note

The iShield Key PQ MUST be connected via USB in order to create or use ML-DSA credentials.

If an RP wants to create an ML-DSA based credential on the iShield Key PQ, the `navigator.credential.create()` requests `CredentialCreationOptions.publicKey.pubKeyCredParams` SHALL include a `PublicKeyCredentialParameters` entry with `alg` being set to an ML-DSA COSE identifier (-48, -49, -50) BEFORE a possible ES256 (-7) entry.

An example for such a request is given below.

The `AuthenticatorAttestationResponse` interfaces `attestationObject` will contain an ML-DSA public key inside the COSE_Key encoded CREDENTIAL PUBLIC KEY in the “authData” field. The exact parameters of this key are as requested by the RP, and also encoded within the COSE_Key object. The `attestationObject` also contains an ML-DSA signature inside the “attStmt” field, which is based on the algorithms stated in the respective “alg” field.

This signature is currently generated with the ML-DSA key pair encoded in the “authData” field (self-attestation).

`navigator.credentials.get()`

Note

The iShield Key PQ MUST be connected via USB in order to create or use ML-DSA credentials.

If an RP wants to get an assertion with a credential that is based on an ML-DSA key pair, it does not need to change anything about its request when compared to any other credentials.

The response to such an assertion request is the same as for any other credential as well, but the signature is created with an ML-DSA private key with the parameter set given during the credential creation.

Examples

`navigator.credentials.create()`

```
await navigator.credentials.create({
  publicKey: {
    attestation: "direct",
    authenticatorSelection: {
      "residentKey": "preferred",
      "userVerification": "required",
      "authenticatorAttachment": "cross-platform",
      "requireResidentKey": false
    },
    challenge: base64URLStringToBuffer("ZVPYLLg1bfTGuatWaWAUGrYBb6zr_N5U
rstWlmRNTs"),
    rp: {
      "name": "Your Relying Party Name",
      "id": "www.your-rp-domain.com"
    },
    user: {
      name: "test_user_123",
      displayName: "Test User 123",
      id: base64URLStringToBuffer("6u0lhT4RWQeEMkP5HVssgqaBey0cNxBHLGHE
-oJcasw"),
    },
    timeout: 60000,
    pubKeyCredParams: [
      {
        "alg": -49,
        "type": "public-key"
      },
      {
        "alg": -7,
        "type": "public-key"
      }
    ],
  }
});
```

With the function `base64URLStringToBuffer` defined as follows.

```
function base64URLStringToBuffer(base64URLString) {  
  // Convert from Base64URL to Base64  
  const base64 = base64URLString.replace(/-/g, '+').replace(/_/g, '/');  
  
  // Pad with '=' until it's a multiple of four  
  const padLength = (4 - (base64.length % 4)) % 4;  
  const padded = base64.padEnd(base64.length + padLength, '=');  
  
  // Convert to a binary string  
  const binary = atob(padded);  
  
  // Convert binary string to buffer  
  const buffer = new ArrayBuffer(binary.length);  
  const bytes = new Uint8Array(buffer);  
  
  for (let i = 0; i < binary.length; i++) {  
    bytes[i] = binary.charCodeAt(i);  
  }  
  
  return buffer;  
}
```